CPU resources as much, and gave programmers relief while they waited for the rise of higher-level languages and their compilers.

History Lesson, II—Enough Is Too Much

With the arrival of languages like C, programming trends started shifting away from the ungainly assembly languages to the more elegant (by those standards, anyway) higher-level languages—which used compilers that translated code to assembly language. As it turned out, these compilers weren't exploiting the potential that CISC had to offer—mostly because compiler designers were at a loss to figure out which instruction to use when. The 8086 had 32 different "Jump If..." instructions to choose from, and more were added with every new processor. The time and effort that went into designing CISC processors started to seem... futile.

The processor design community split into two schools of thought—one concentrated on developing better compilers for CISC processors (notably the Intel x86, which is all-pervasive today), while the other believed that CISC processors were grossly overdesigned, and a simpler, more compiler-programmer-friendly instruction set was in order. The idea came to be known as Reduced Instruction Set Computing (RISC), and came with its own set of advantages and disadvantages.

Every RISC instruction takes one single clock cycle to execute, so the CPU never has to wait for complex instructions to finish executing; the time saved is even more prominent when instructions are sent alternately to multiple processors. Most operations would take the same time on both RISC and CISC processors regardless, but the hardware required to decode RISC instructions is less complex, which makes it easier and cheaper to implement. The most significant disadvantage of the RISC architecture, however, is that it relies quite heavily on software—a badly written compiler can have a devastating effect on performance, as could badly-written programs.

In a nutshell, the simplicity of implementing an RISC machine makes it perfect for mobile processors, so that's precisely what Acorn decided to use when they came up with the ARM.

At The Drawing Board

The biggest thing to consider when designing a mobile processor is power consumption. Every single decision that follows—architecture, clock speed and so on—is driven by the need to use as little power as possible, and naturally so. How would you like it if you had to keep recharging your new PDA phone every three hours? Keeping power consumption low also ensures that less of it is dissipated as heat—which is good, because you don't want to be saddled with a heat-sink in your pocket. Or worse.

It's also important that these processors cost very little, which in turn means that they should be easy to design.

So in 1983, Acorn Computers designed the ARM (Acorn RISC Machine), a 32-bit RISC processor which had a third of the transistors as the Motorola 68000—which was six years older than the ARM—and could still actually do stuff.

Taking More RISCs

Consider this bit of code that calculates the GCD (Greatest Common Divisor) for the numbers i and j :

```
while (i != j)
    if (i > j)
        i -= j;
    else
        j -= i;
return i;
```

The premise is that if the lower of i and j is subtracted from the greater perpetually till they're finally equal, the final number is the GCD.

When this code is compiled to x86 assembly, you'll have instructions that:

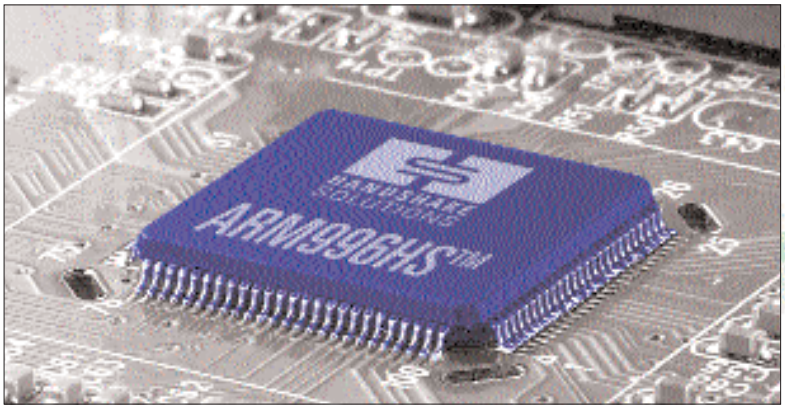
1. Check the equality of i and j; the CPU will execute the following code if they're equal, or will start executing code from another (specified) memory location if not. This is called *branching*—when there two possible outcomes of the same situation.

2. Once inside the loop, check which of i and j is greater and subtract the lower from it—resulting in another possible branch.

The x86 processor has an instruction pipeline, which loads instructions from the system's memory and keeps them ready for the CPU to execute. In the above case, let's say that while the CPU is performing the first comparison, the code for the rest of the loop is already in the pipeline. If i and j are equal, however, all that code is unnecessary, so the pipeline has to be cleared, and instructions need to be loaded from a new memory location all over again. This wastes the pipeline, CPU time; as we've mentioned before, these are expensive resources, so wastes are unacceptable. The x86 architecture compensates for this with a *branch predictor*, which does exactly what the name implies. Today's predictors are accurate around 99 per cent of the time, and the overall gains offset the losses caused by the one percent.

ARM, however, would have none of this. Branch predictors add complexity, so Acorn decided to use the last four bits of their instruc-

The most significant disadvantage of the RISC architecture is that it relies quite heavily on software



Anyone can make an ARM processor—once you've paid Acorn for the license, that is